SOLAR▸SWARM

Technical Reference
Martin Pham | Perth, AU

Capital & Risk Management

Kelly Criterion Formula

File: `capital/kelly_engine.py`

The system uses 25% Fractional Kelly to size experiment budgets. This reduces variance while preserving the mathematical edge of full Kelly.

Full Kelly formula

```
f* = (b × p − q) / b

where:
  b = odds (expected ROI ratio, default 3.0 — 3× payoff)
  p = probability of winning
  q = probability of losing = (1 − p)
```

25% Fractional Kelly

```
f_actual = f* × 0.25
budget   = f_actual × WEEKLY_BUDGET_AUD
budget   = min(budget, WEEKLY_BUDGET_AUD × 0.25)  # hard cap: never > 25% per experiment
```

Confidence to Win Probability Mapping

The 4-component confidence score (0–10) is mapped to a win probability before applying Kelly:

```
p = 0.2 + (confidence_score / 10.0 × 0.65)
p = clamp(p, 0.10, 0.90)

# Examples:
# confidence 5.0 → p = 0.20 + 0.325 = 0.525 (52.5% win probability)
# confidence 7.0 → p = 0.20 + 0.455 = 0.655 (65.5%)
# confidence 9.0 → p = 0.20 + 0.585 = 0.785 (78.5%)
```

4-Component Confidence Scoring

```
score_experiment(market_signal, competitive_gap, execution_speed, revenue_path)
# Each component rated 0–10, averaged:
confidence_score = (market_signal + competitive_gap + execution_speed + revenue_path) / 4
```

COMPONENT	WHAT IT MEASURES
`market_signal`	Evidence of real demand for this experiment
`competitive_gap`	How underserved this opportunity is
`execution_speed`	Can this be tested within 72 hours?
`revenue_path`	Is there a clear, direct line to cash?

Example Calculation

```
confidence_score = 7.5
b = 3.0 (default win multiplier)
p = 0.2 + (7.5/10 × 0.65) = 0.6875
q = 1 − 0.6875 = 0.3125
f* = (3.0 × 0.6875 − 0.3125) / 3.0 = 0.5833
f_actual = 0.5833 × 0.25 = 0.1458
budget = 0.1458 × $500 = $72.92
# Hard cap check: $500 × 0.25 = $125 → not triggered
# Budget allocated: $72.92 AUD
```

Budget Buckets

Configured in `config.py`, applied in `capital/portfolio_manager.py`.

BUCKET	ALLOCATION	PURPOSE
`exploit`	60%	Proven strategies; scale what works
`explore`	30%	72-hour test experiments; learn fast
`moonshot`	10%	High-risk, high-reward; long shots only

Bucket assignment rules (`assign_bucket()`):

```
if devil_score >= 7 and confidence_score >= 7:
    return "moonshot"   # Risky but high confidence
elif confidence_score >= 8 and devil_score < 5:
    return "exploit"    # Proven, low risk
else:
    return "explore"    # Default: test it first
```

Checking available budget:

```
get_bucket_budgets()   # {exploit: 300, explore: 150, moonshot: 50, total: 500}
get_bucket_usage()     # Spent this week per bucket
get_bucket_remaining() # Remaining per bucket
can_allocate(bucket, amount)  # True if enough remaining
```

Circuit Breaker States

File: `capital/circuit_breaker.py`

The circuit breaker monitors experiment outcomes and budget burn. It triggers automatic halts before losses compound.

LEVEL	TRIGGER CONDITION	ACTION
Green	All clear	Normal operation
Yellow	3 consecutive experiment failures	Warning logged; system continues
Orange	Budget burn rate > 150% of plan	Warning logged; system slows
Red	5+ consecutive failures OR single loss > 40% of weekly budget	Full halt; General cycle suspended

Threshold values (from `config.py`):

```
CB_YELLOW_FAILURES = 3
CB_ORANGE_BURN_RATE = 1.50      # 150% of planned daily spend
CB_RED_FAILURES = 5
CB_RED_LOSS_FRACTION = 0.40     # 40% of WEEKLY_BUDGET_AUD
```

Checking state:

```
is_halted()                     # True if Red
get_current_level()             # 'green'/'yellow'/'orange'/'red'
get_breaker_history(limit=20)   # Recent events from circuit_breaker_log
```

Resetting a Red breaker:

A Red circuit breaker requires explicit human reset:

```
curl -X POST http://localhost:5000/approve-breaker \
  -H "Content-Type: application/json" \
  -d '{"approved_by": "martin"}'
```

Or via the Python API:

```
from capital.circuit_breaker import reset_breaker
reset_breaker("martin")
```

Budget burn rate calculation:

```
days_elapsed = max(1, datetime.utcnow().weekday() + 1)  # 1–7
planned_to_date = (WEEKLY_BUDGET_AUD / 7) × days_elapsed
burn_rate = actual_spent / planned_to_date
# burn_rate > 1.5 → Orange trigger
```

Confidence Routing Thresholds

After the 4-component score and red team adjustment:

SCORE	ROUTE	ACTION
> 8.5	`auto_proceed`	Experiment approved immediately, budget allocated
5.0–8.5	`human_gate`	Slack alert sent; human must approve or reject
< 5.0	`auto_kill`	Experiment killed immediately, no budget allocated

Red team downgrade: If `devil_score > 6`, confidence is reduced:

```
penalty = (devil_score − 6) × 0.5
adjusted_confidence = original_confidence − penalty
```

Example: confidence 8.2, devil_score 7 → penalty 0.5 → adjusted 7.7 → human_gate

---

72-Hour Explore Protocol

File: `capital/portfolio_manager.py` — `get_explore_phase()` and `run_explore_monitor()`

All experiments in the `explore` bucket follow a structured 72-hour lifecycle. The `explore_monitor` scheduler job runs every 2 hours to advance phases.

TIME	PHASE	REQUIRED ACTION
0–12h	`asset_creation`	Build landing page, ad copy, creatives
12–24h	`distribution`	Organic channels only — post, email, LinkedIn
24–48h	`signal_observation`	Observe CTR, engagement, enquiries — no spend
48–60h	`decision_point`	If CTR ≥ 2%: activate paid spend; else organic only
60–72h	`final_assessment`	Promote to exploit bucket or kill and log learnings
>72h	`expired`	Auto-killed; mutation engine generates variants on Monday

CTR threshold for paid spend:

```
EXPLORE_CTR_THRESHOLD = 0.02  # 2%
```

If CTR at the decision_point is ≥ 2%, `activate_paid_spend(experiment_id)` is called. This sets `paid_spend_activated=1` on the experiment record and sends a Slack notification.

Phase transitions post Slack notifications automatically when the explore_monitor detects a phase change.

Human Gate Approval Flow

When an experiment is routed to `human_gate` (confidence 5.0–8.5):

1. Experiment saved with `status='pending'` in `experiments` table
2. `alert_human_gate()` posts a Slack message with **Approve** and **Reject** buttons
3. Human clicks button in Slack → Slack sends POST to `/slack/actions`
4. System calls `_approve()` or `_reject()` and updates the message
5. Alternatively, human uses the REST API directly:

```
# Approve
curl -X POST http://localhost:5000/approve/42

# Reject
curl -X POST http://localhost:5000/reject/42 \
  -H "Content-Type: application/json" \
  -d '{"reason": "Budget too tight this week"}'

# See all pending
curl http://localhost:5000/pending
```

On approval: `status` → `approved`, `budget_allocated` calculated, `approved_by` and `approved_at` stamped. Cold ledger entry written with `human_involved=True`.

Pheromone Decay Algorithm

Pheromone signals guide the General toward experiment types that have worked before. Signals degrade over time to prevent stale information from dominating.

Decay parameters:

```
PHEROMONE_DECAY_DAYS = 7     # Signals stay at full strength for 7 days
PHEROMONE_DECAY_RATE = 0.50  # 50% weight loss per decay application
```

Decay formula (applied daily at midnight UTC):

```
new_decay_factor = current_decay_factor × (1.0 − PHEROMONE_DECAY_RATE)
# = current × 0.50

# Signals with decay_factor < 0.01 are treated as expired (invisible to agents)
```

Effect: A signal created 7 days ago has `decay_factor = 1.0`. After the first decay application, `0.5`. After the second, `0.25`. After three days of decay: `0.125`. After ~7 decay days from creation: effectively zero.

Reading active pheromones:

```
from memory.hot_memory import get_active_pheromones
signals = get_active_pheromones(topic="solar_prospecting")  # topic optional
# Returns signals where decay_factor > 0.01, sorted by strength DESC
```